



QuizArena

Online kvíz- és vetélkedőplatform

Szoftverdokumentáció

Képzés: Szoftverfejlesztő és -tesztelő (OKTÁV Technikum és Szakképző Iskola)

Készítette: Pintér Szabolcs, Perger Balázs

GitHub: github.com/pisabolcs/Vizsgaremek_QuizArena

2026. szeptember

Technológiák: Node.js · Express · SQLite (better-sqlite3) · HTML5 · CSS3 · natív JavaScript

Tartalomjegyzék

Tartalomjegyzék	2
1. Bevezetés.....	3
2. Fejlesztési dokumentáció	4
2.1 Fejlesztői környezet leírása	4
2.2 Választott technológia	4
2.3 Használt technológiák (kiegészítés).....	4
2.4 Az adatbázis felépítése	5
2.5 Egy kiválasztott rész kódba menő leírása	6
2.5.1 A helyes válasz sosem kerül a böngészőbe idő előtt.....	6
2.5.2 A pontszámítás tiszta, tesztelhető függvényekben	6
2.5.3 Fő API-végpontok	7
2.6 Tesztelési dokumentáció	7
3. Felhasználói dokumentáció	8
3.1 Mi szükséges az alkalmazás megtekintéséhez?.....	8
3.2 Hogyan indítható az alkalmazás?	8
3.3 Képes bemutató	8
4. Összefoglalás	15
4.1 Mit sikerült elérni	15
4.2 Tervezett továbbfejlesztések.....	15
4.3 Csapatmunka és munkamegosztás.....	15
4.4 Zárszó.....	15

1. Bevezetés

A QuizArena egy böngészőben futó kvíz- és vetélkedőplatform: a felhasználók regisztrálnak, kvizeket töltenek ki, XP-t gyűjtenek, szintet lépnek, és versenyeznek egymással egy ranglistán. A projekt a Szoftverfejlesztő és -tesztelő képzés vizsgaremekeként készült, Pintér Szabolcs és Perger Balázs közös munkájával.

A témát azért választottuk, mert egy kvízalkalmazás köré viszonylag könnyen felépíthető egy valódi, több táblás relációs adatmodell és érdemi serveroldali üzleti logika (pontszámítás, időmérés, szintlépés, ranglista-aggregáció), miközben a végeredmény bárki számára azonnal kipróbálható és élvezhető — nem csak fejlesztőknek szóló, elvont demó.

Az alkalmazást elsősorban egyéni felhasználóknak szántuk, akik szórakozásból vagy tudáspróbaként töltenek ki kvizeket saját tempójukban, időre, vagy a fokozódó nehézségű, segítségekkel támogatott „Legyen Ön is milliomos” módban. A rendszer emellett előkészíti a terepet közösségi használatra is (kvízesték, baráti kihívások), ezekről a 5. fejezet ír bővebben.

A tervezett fő funkciók: felhasználói fiókkezelés, kategóriákba és nehézségi szintekbe rendezett kérdésbank, egyedi kérdéssorok (kvizek) összeállítása, három játékmód serveroldali, csalásbiztos pontszámítással, XP- és szintrendszer, valamint ranglista. Ezek közül melyik mennyire készült el, azt a 5.1 fejezet (Összefoglalás) foglalja össze.

2. Fejlesztési dokumentáció

2.1 Fejlesztői környezet leírása

A projekt egyetlen Node.js folyamatként fut: nincs külön webservert (Apache/nginx) vagy külön adatbázis-szerver a fejlesztői gépen — a Node.js beépített HTTP-szervere (Express-en keresztül) szolgálja ki mind az API-t, mind a statikus frontend fájlokat, az adatbázis pedig egyetlen fájl (SQLite).

Terület	Használt szoftver
Kódszerkesztő	Visual Studio Code
Futtatókörnyezet	Node.js (fejlesztéskor: v22 LTS)
Csomagkezelő	npm
"Szerver" a fejlesztői gépen	a Node.js/Express folyamat maga (nincs külön Apache/nginx)
Adatbázis a fejlesztői gépen	SQLite fájl (better-sqlite3 könyvtáron keresztül), adatbázis-szerver telepítése nélkül
Verziókezelés	Git, GitHub (github.com/piszabolcs/Vizsgaremek_QuizArena)
Böngésző / tesztelés	Google Chrome, Chrome DevTools (hálózati kérések, reszponzív nézet ellenőrzése)

2.2 Választott technológia

Backend: Node.js + Express 4, kliens-szerver, REST elvű felépítésben. Adatbázis: SQLite (better-sqlite3 könyvtár). Frontend: natív HTML5, CSS3 és JavaScript, build-lépés és keretrendszer (React, Vue stb.) nélkül — a frontend a böngésző beépített Fetch API-jával kommunikál a backenddel.

Ez a választás tudatos egyszerűsítés: a feladat nem indokolta egy nehézsúlyú frontend keretrendszer bevezetését, és így a teljes forráskód — build-eszközök és transzpilálás nélkül — bármikor közvetlenül olvasható és futtatható marad.

2.3 Használt technológiák (kiegészítés)

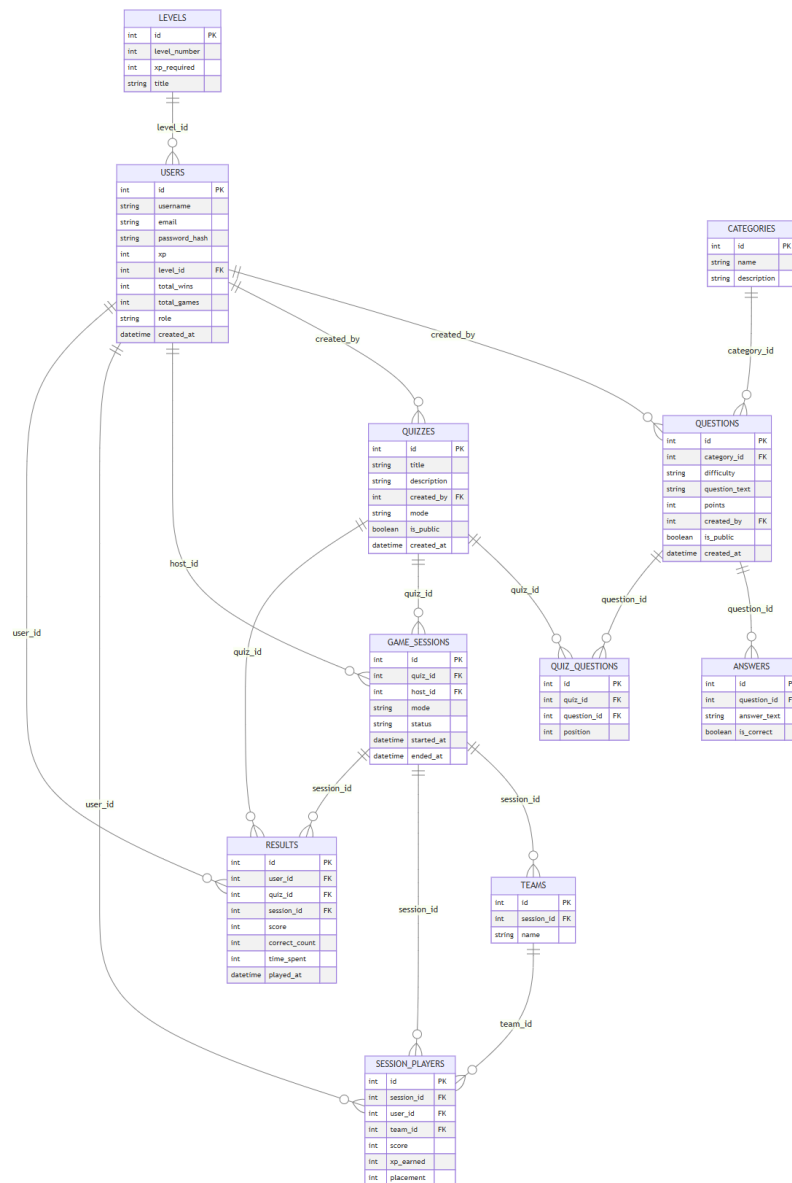
Az alap html-css-js + Node.js/Express + SQLite váz mellett a projekt az alábbi kiegészítő könyvtárakat használja:

Könyvtár	Szerepe
better-sqlite3	szinkron, natív SQLite driver — egyszerűbb és gyorsabb, mint egy aszinkron driver, mert a lekérdezések itt nem igényelnek Promise-láncot
express-session	szerveroldali munkamenet-kezelés (bejelentkezési állapot)
node:test (beépített)	a Node.js saját tesztfuttatója az egységtesztekhez, külön tesztkeretrendszer (pl. Jest) telepítése nélkül

Frontend-oldali keretrendszert (pl. Bootstrap, React) szándékosan nem használtunk — ezzel a döntéssel a hangsúlyt a backend üzleti logikára és az adatmodellre helyeztük.

2.4 Az adatbázis felépítése

Az adatmodell 11 táblából áll, idegen kulcsokkal összekapcsolva. Az alábbi ábra a teljes adatbázist mutatja: minden tábla az oszlopaival és a köztük lévő kapcsolatokkal.



1. ábra — a QuizArena adatmodelljének ER-diagramja (11 tábla, teljes kapcsolatrendszer)

Néhány kulcsfontosságú kapcsolat:

- `users.level_id` → `levels.id`: minden felhasználóhoz pontosan egy aktuális szint tartozik;
- `questions.category_id` → `categories.id` és `answers.question_id` → `questions.id`: a kérdésbank kategóriákba rendezett, minden kérdéshez több válaszlehetőség tartozik;
- `quiz_questions`: kapcsolótábla a quizzes és a questions táblák között (sok-a-sokhoz), amely a sorrendet (position) is tárolja;
- `results.user_id` / `results.quiz_id` / `results.session_id`: minden kitöltés eredménye visszavezethető arra, ki, melyik kvízt, esetleg melyik játékmenetben töltötte ki.

A teljes DDL (adattípusokkal, kulcsokkal) az adatbázis/schema.sql fájlban, a kezdőadatok pedig az adatbázis/seed.sql fájlban találhatók — ezek a repository adatbázis/ mappájának részei.

2.5 Egy kiválasztott rész kódba menő leírása

(Ez a rész a tanárral egyeztetve, közösen választható ki — az alábbi a saját javaslatunk: a szerveroldali, csalásbiztos pontszámítás és válaszkezelés, mert ez az alkalmazás legkritikusabb üzleti logikája.)

2.5.1 A helyes válasz sosem kerül a böngészőbe idő előtt

Az egyik legfontosabb üzleti szabály, hogy a böngésző a kérdés megválaszolása előtt soha nem kaphatja meg, melyik a helyes válasz — különben a fejlesztői konzol Network fülén ki lehetne olvasni a megoldást. Ezért a kvíz lekérdezésekor a szerver szándékosan kihagyja az `is_correct` mezőt a válaszból, és összekeveri a kérdések, illetve válaszlehetőségek sorrendjét:

```
// EGY KVIZ KERDESEI SORRENDEN, A VALASZOKKAL EGYUTT
// fontos: az is_correct mezot itt szandekosan NEM kuldjuk el, hogy ne lehessen
// a fejlesztői konzolból kiolvasni a helyes valaszt meg mielőtt valaszolnánk
router.get("/:id", function (req, res) {
  ...
  for (let i = 0; i < kerdesek.length; i++) {
    let valaszok = db.prepare(
      "SELECT id, answer_text FROM answers WHERE question_id = ?"
    ).all(kerdesek[i].id);
    tombKeveres(valaszok); // Fisher-Yates keveres
    kerdesek[i].valaszok = valaszok;
  }
  if (kviz.mode !== "millionaire") {
    tombKeveres(kerdesek); // milliomos modnal a nehezedes a sorrend, azt nem keverjuk
  }
  ...
});
```

2.5.2 A pontszámítás tiszta, tesztelhető függvényekben

A pontszámítás — szintén a vizsgafeladat előírása szerint — teljes egészében a szerveren történik, egy önálló modulban (`server/utils/scoring.js`), amely szándékosan nem nyúl az adatbázishoz vagy a HTTP-kéréshez: csak bemenetet kap és értéket ad vissza. Ez azért fontos, mert így a logika egyszerű, gyors egységtesztekkel ellenőrizhető (lásd a Tesztelési dokumentációt).

```
// jo valasz eseten a kerdes pontjat kapja (idore menonel + ido-bonusszal),
// rossz valasz eseten a kerdes pontjanak fele levonasra kerul
function pontEgyKerdesre(helyesE, kerdesPont, idoreMegy, hatralevoMasodperc) {
  if (helyesE === false) {
    return -(kerdesPont / 2);
  }
  let pont = kerdesPont;
  if (idoreMegy === true) {
    let bonus = hatralevoMasodperc;
    if (bonus > 10) bonus = 10;
    if (bonus < 0) bonus = 0;
    pont = pont + bonus;
  }
  return pont;
}

// a vegeredmeny sose lehet negativ, meg sok buntetes eseten se
function kvizOsszpont(valaszok) {
```

```

let osszeg = 0;
for (let i = 0; i < valaszok.length; i++) {
  let v = valaszok[i];
  osszeg = osszeg + pontEgyKerdesre(v.helyes, v.pont, v.idoreMegy, v.ido);
}
if (osszeg < 0) osszeg = 0;
return osszeg;
}

```

A rossz válasz nem 0, hanem a kérdés pontértékének mínusz felét éri — ez visszatartja a találgatást —, ugyanakkor a végeredmény sosem mehet 0 alá, hogy ne legyen félrevezető. A kiértékeléskor (POST /api/games/submit) a szerver a beküldött válasz-azonosítót mindig újra lekérdezi az adatbázisból, és ellenőrzi, hogy az valóban az adott kérdéshez tartozik-e — a kliens tehát elméletben sem tudna hamis pontszámot beküldeni.

2.5.3 Fő API-végpontok

Végpont	Leírás
POST /api/auth/register, /login, /logout	regisztráció, bejelentkezés, kijelentkezés
GET /api/quizzes, GET /api/quizzes/:id	kvizek listázása, egy kvíz kérdései kevert sorrendben
POST /api/games/submit	kitöltött kvíz beküldése, szerveroldali kiértékelés
POST /api/games/hint/fifty-fifty, /audience, /swap	a milliomos mód segítségével
GET /api/leaderboard	ranglista lekérése
GET /api/users/me, /me/results, /me/quizzes	a bejelentkezett felhasználó profilja, előzményei, saját kvizei

2.6 Tesztelési dokumentáció

A tesztelést — a mintának megfelelően — külön dokumentumban részletezzük: lásd a dokumentum/QuizArena_tesztelési_dokumentacio.pdf fájlt. Ez tartalmazza a tesztelési stratégiát, az automata egységtesztek és a manuális teszteseteket táblázatos formában, a hibajegyzéket és a tesztelési jegyzőkönyvet.

3. Felhasználói dokumentáció

3.1 Mi szükséges az alkalmazás megtekintéséhez?

Az alkalmazás futtatásához nincs szükség adatbázis-szerverre vagy webszerverre telepítésére — kizárólag Node.js-re (amely tartalmazza az npm csomagkezelőt is) és egy modern böngészőre (pl. Chrome, Edge, Firefox).

- Node.js (18-as vagy újabb LTS verzió ajánlott)
- egy tetszőleges modern böngésző
- nincs szükség MySQL/Postgres stb. telepítésére — az adatbázis egyetlen SQLite fájl

Aki nem szeretne Node.js-t telepíteni, a repository telepito_portable/ mappájában egy hordozható, előre becsomagolt verziót is talál, amely önmagában, telepítés nélkül elindítható (lásd a telepito_portable/README.txt fájlt).

3.2 Hogyan indítható az alkalmazás?

```
cd forraskod
npm install      # fuggosegek telepítése
npm run seed     # adatbázis felepítése és feltöltése mintaadatokkal
npm start        # szerver indítása: http://localhost:3000
```

Ezután a `http://localhost:3000` címen érhető el az alkalmazás főoldala. Az `npm run seed` csak akkor futtatható, ha a szerver éppen nem fut (Windows zárolja a nyitva lévő adatbázis-fájlt) — előbb `Ctrl+C`-vel állítsd le, utána futtasd a seedet.

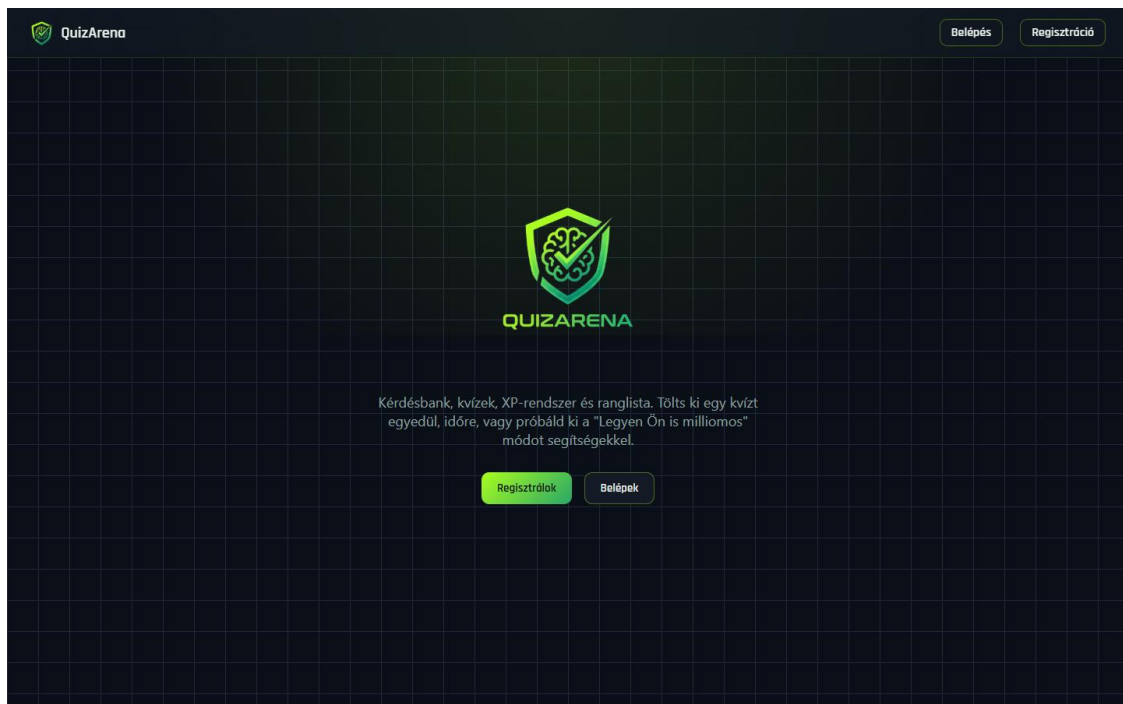
A főoldalon (kijelentkezett állapotban) csak egy bemutatkozó felület és a regisztráció/belépés gombok érhetők el; a kvizek, a ranglista és a profil bejelentkezés után nyílnak meg. Az alábbi teszt fiókokkal azonnal ki lehet próbálni az alkalmazást, saját regisztráció nélkül is:

Email	Jelszó	Szerepkör
admin@quizarena.hu	admin123	admin
host@quizarena.hu	host123	host
anna@example.com	jelszo1	player
bela@example.com	jelszo2	player
cili@example.com	jelszo3	player

3.3 Képes bemutató

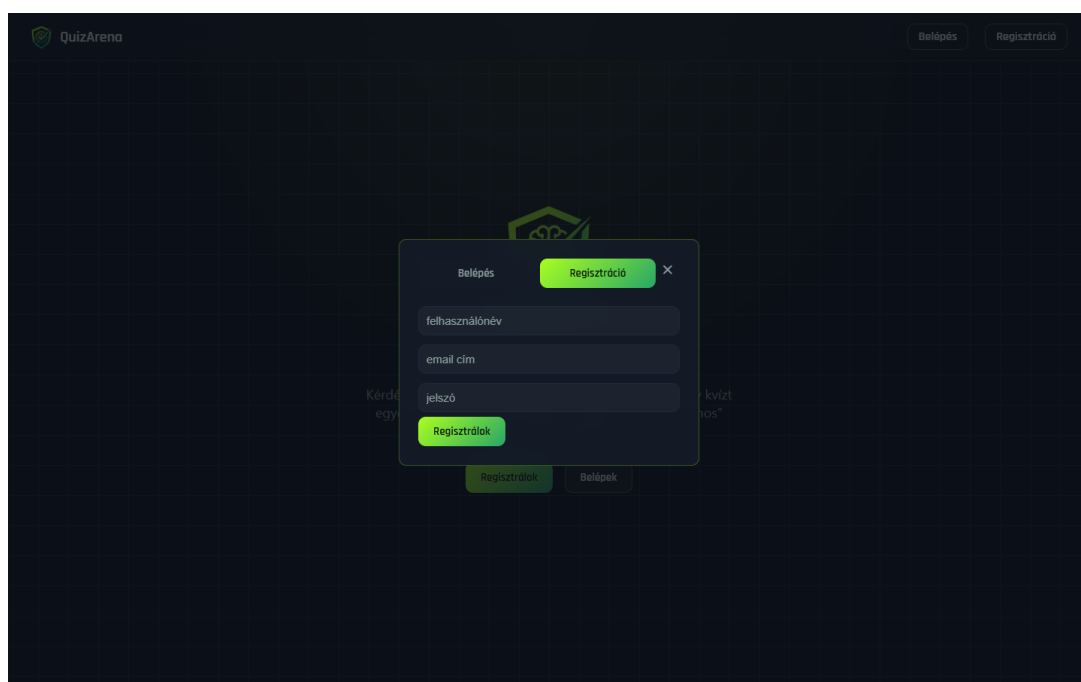
Ez a rész az alkalmazás fő képernyőit mutatja be, sorban végigvezetve egy tipikus felhasználói élményen — a főoldaltól a kvíz kitöltésén át az eredményig.

Kijelentkezett állapotban egy bemutatkozó (hero) felület fogadja a látogatót, ahonnan regisztrálni vagy bejelentkezni lehet.



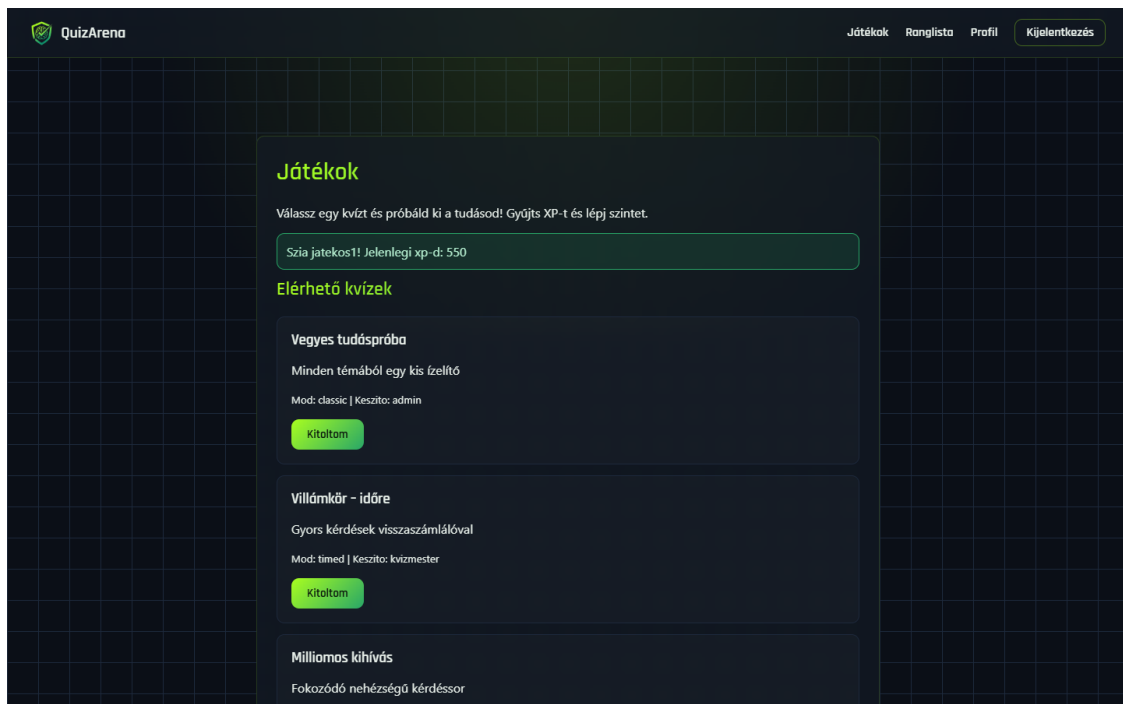
2. ábra — a főoldal kijelentkezett állapotban

A regisztráció és a bejelentkezés egy közös, lapváltós felugró ablakban történik, oldaltöltés nélkül.



3. ábra — regisztrációs űrlap felugró ablakban

Bejelentkezés után a Játékok oldal listázza az elérhető kvízeket, feltüntetve a játékmódot és a készítőit.



4. ábra — az elérhető kvízek listája

Klasszikus módban az összes kérdés egyszerre látszik; a választott válasz kiemelődik, majd egyetlen gombnyomással adható be a teljes kitöltés.

Vegyes tudóspróba

Minden kérdésből egy kis bónusz!

1. Péter születe

Ady Endre
Arany János
József Attila

2. Hány napból áll egy századvé?

365
366
367
368

3. Hány játékos van egy focicsapatban a pályán egyszerre?

10
11
12
13

4. Mi Magyarországi Hódolom?

Nem tudom
Béni
Béni
Béni

5. Mi a víz kémiai képlete?

H₂O
H₂
H₂O₂
CO₂

6. Mit jelent a CPU rövidítés?

Processzor
Számítógépes tároló
Készen állsz a feladatra?
Béni

7. Melyik animációs stúdió készítette az Óz varázslókirályt?

Béni
Béni
Béni
Béni

8. Melyik évszázad alapították a Magyar Államot szent István királjukkal?

1000
1001
1002
1003

9. Ki fedezte fel a Mars Látót?

Nem tudom
Béni
Béni
Béni

10. Hány napja van egy hagyományos gőzmozgó?

1
2
3
4

Vissza a kezdőhöz

5. ábra — klasszikus mód: minden kérdés egy oldalon

Beadás után a szervertoldalon kiszámolt eredmény azonnal megjelenik: helyes válaszok aránya, szerzett pont, kapott és összesített XP, valamint hogy a játékos nyert-e.

Eredmény

Helyes válaszok: 10 / 10

Szerzett pont: 100

Kapott xp: 100

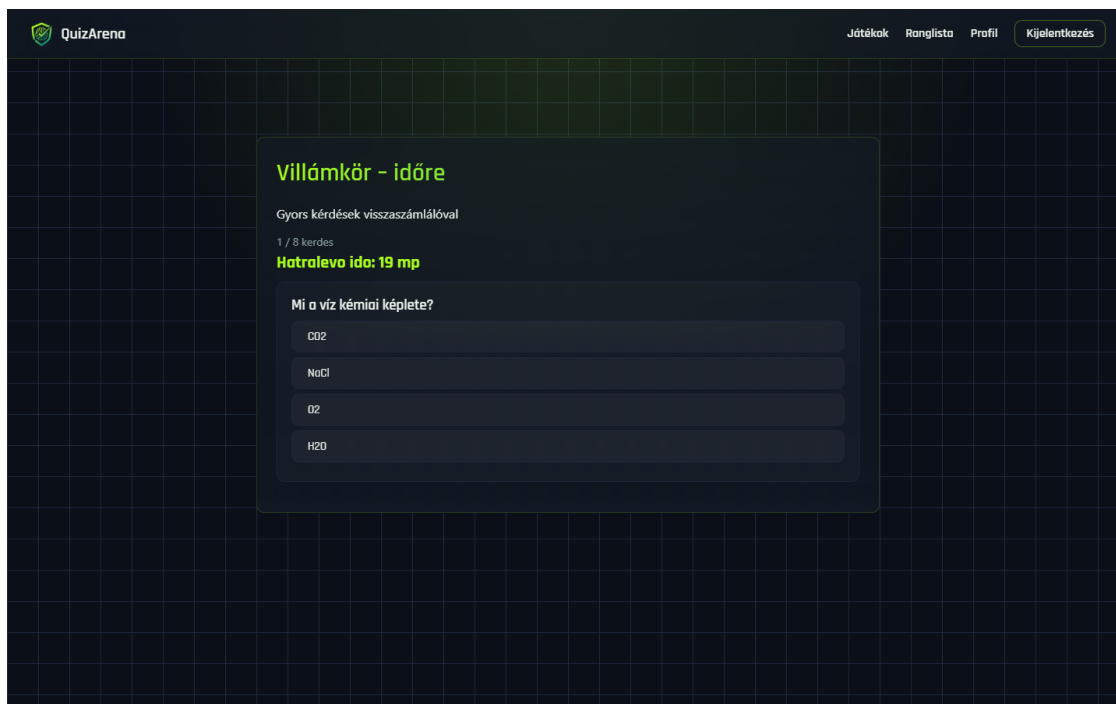
Összes xp most: 650

Jelenlegi szint: 4

Gratulálok, nyertel!

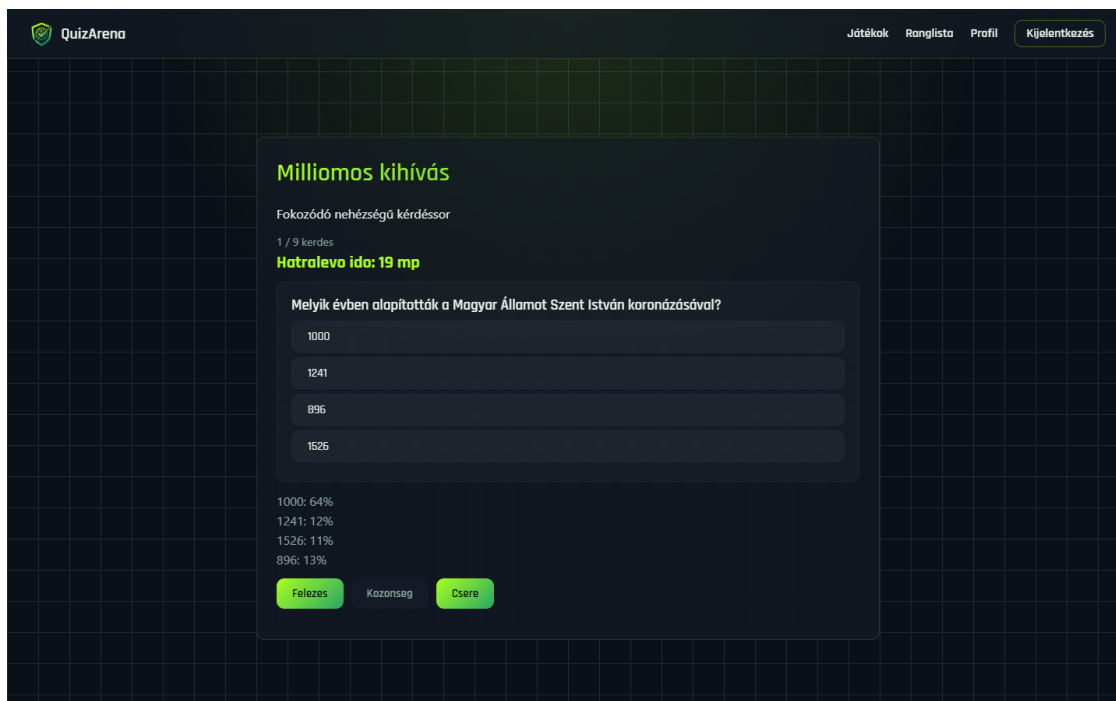
6. ábra — eredmény-doboz beadás után (10/10 helyes válasz)

Időre menő módban a kérdések egyesével jönnek, mindegyikre 20 másodperc jár; ha lejár az idő, a rendszer üres válaszként veszi és továbblép.



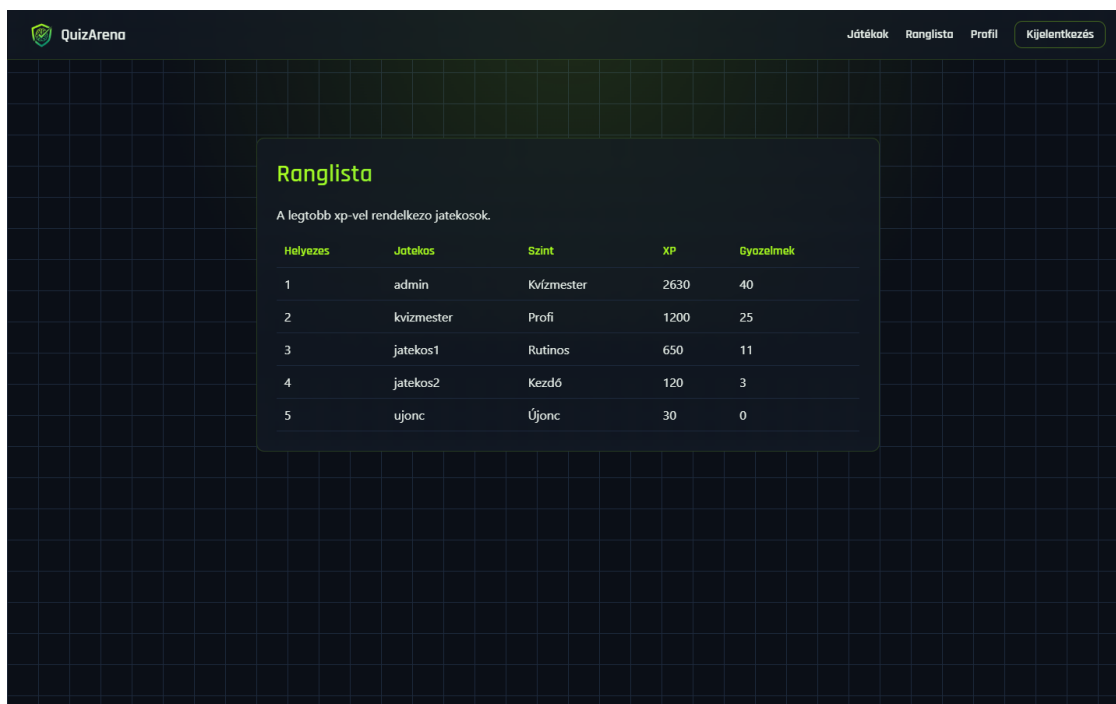
7. ábra — időre menő mód, futó visszaszámlálással

A „Legyen Ön is milliomos” mód fokozódó nehézségű kérdéssorral, azonnali kieséssel és három, egyszer használható segítséggel (felezés, közönség, csere) dolgozik.



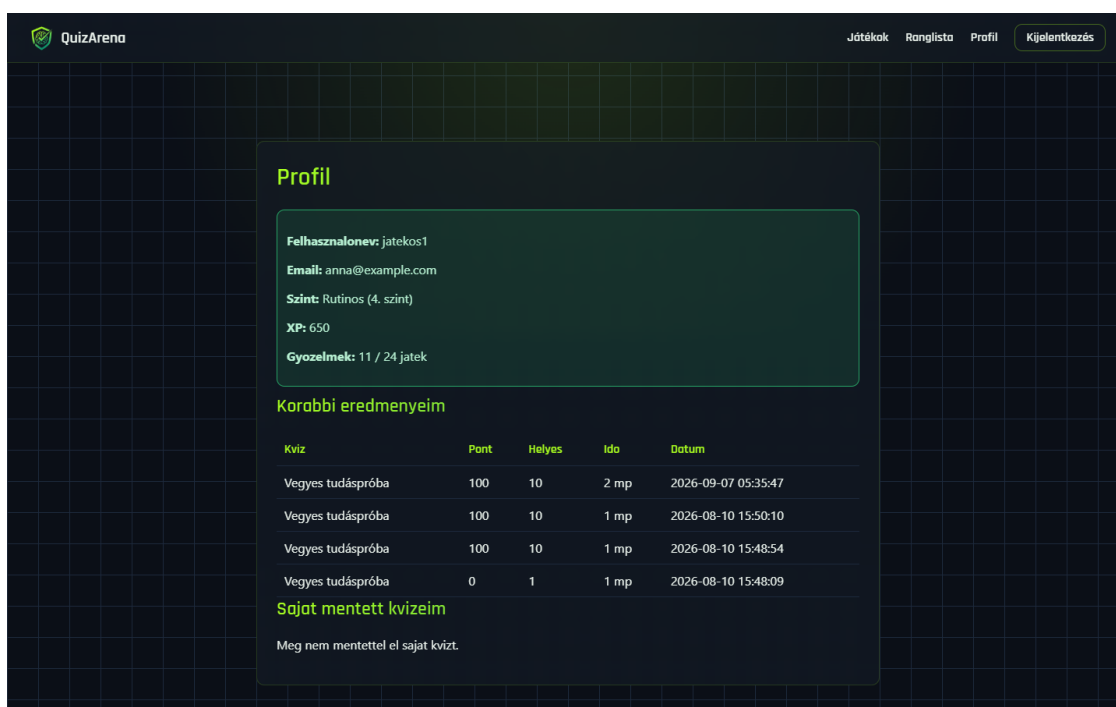
8. ábra — milliomos mód a segítség-sávval és a közönség-szavazattal

A Ranglista oldal a legtöbb XP-vel rendelkező játékosokat listázza, a szintjükkel és győzelmeik számával.



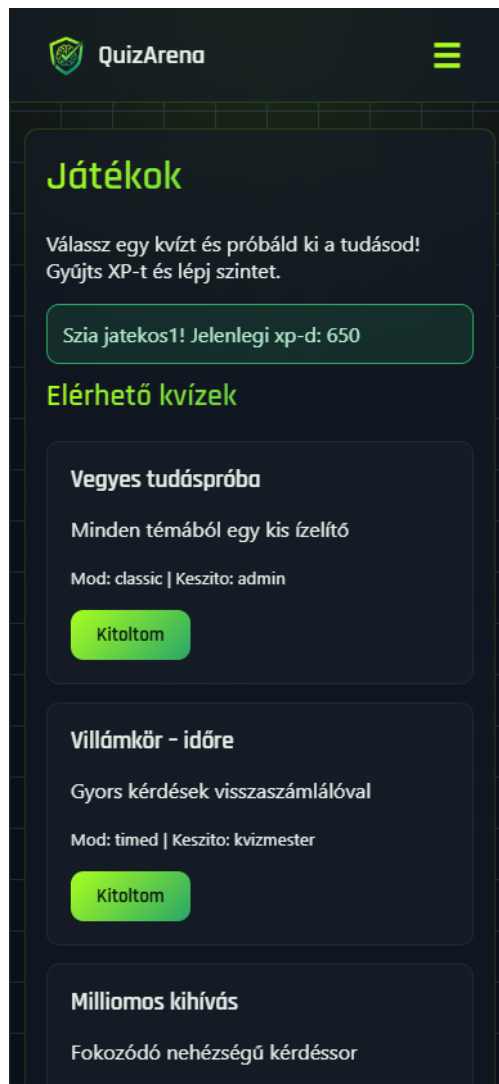
9. ábra — a ranglista

A profil oldalon a saját alapadatok, az aktuális szint és XP, valamint a korábbi kitöltések története látható.



10. ábra — profil oldal a kitöltési előzményekkel

A felület CSS media query-kkel alkalmazkodik kisebb képernyőkhöz is: a navigáció hamburger-menüvé alakul.



11. ábra — a kvízlista mobil nézetben (390 px széles viewport)

4. Összefoglalás

4.1 Mit sikerült elérni

A QuizArena egy stabilan futó, jól strukturált webalkalmazás lett, amely a vizsgafeladat által elvárt funkcionális kört magas színvonalon fedi le. A rendszer három, egymástól jól elkülönülő játékmódot kínál — a klasszikus kitöltéstől az időre menő módon át a fokozódó nehézségű, segítségeket is tartalmazó „Legyen Ön is milliomos” módig —, mindezt serveroldali, csalásbiztos pontszámítással, XP- és szintrendszerrel, valamint ranglistával megtámogatva.

A projekt erőssége a végiggondolt architektúra: a routes/middleware/utils rétegek tiszta elválasztása, a paraméteres lekérdezésekre épülő SQL injection elleni védelem, a session-alapú jogosultságkezelés és a tesztelt, önálló pontszámító modul mind azt mutatják, hogy nem csupán működő, hanem karbantartható kódot írtunk. A kérdésbank 80 kérdést tartalmaz 10 kategóriában, 3 nehézségi szinten, és a rendszer 5 különböző szerepkörű teszt-felhasználóval együtt azonnal kipróbálható.

4.2 Tervezett továbbfejlesztések

A jelenlegi, stabilan működő alapokra építve az alábbi irányok vinnék tovább a platformot:

- Kérdéssor-összeállító felület: a backend már teljes API-t biztosít egyedi kvíz összeállításához (POST /api/quizzes, POST /api/questions), a hozzá tartozó kezelőfelület a következő fejlesztési szakasz feladata;
- Integrációs tesztek bővítése a meglévő egységtesztek mellé (pl. Supertest csomaggal), a fő API-folyamatokra;
- Több fős / csapatos kvízeszt mód kiteljesítése: az adatbázis-modell (game_sessions, teams, session_players) és egy induló végpont már készen áll, a kvízmesteri vezénylő felület a következő mérföldkő;
- „Kihívás egy barátot” funkció, grafikonos fejlődés-statisztika és kitüntetés-/jelvényrendszer bevezetése;
- bcrypt bevezetése a jelenlegi SHA-256 jelszó-hashelés helyett, és a session titkosító kulcs környezeti változóba szervezése éles üzemeltetéshez.

4.3 Csapatmunka és munkamegosztás

[Ezt a részt egyeztetve, a tényleges munkamegosztás alapján töltsétek ki — pl. ki dolgozott inkább a backenden/frontend-en/adatbázis-terven, hogyan egyeztettetek (Git commitok, közös ülések), mi volt jó a közös munkában. Az alábbi egy kiindulási vázlat:]

A fejlesztés két fő területre bontva zajlott: a backend (Express-végpontok, adatbázis-séma, pontszámítási logika) és a frontend (HTML/CSS/JS oldalak, felhasználói élmény) között. A csapat rendszeresen egyeztetett a haladásról, és a Git-en keresztül közösen vitte előre a kódbázist. A közös munka legnagyobb tanulsága [ide írjátok be a saját tapasztalatokat].

4.4 Zárszó

Össességében a QuizArena magabiztosan mutatja be egy teljes körű, adatbázis-alapú webalkalmazás tervezéséhez és megvalósításához szükséges tudást és mérnöki szemléletet — a tervezéstől az adatmodellen és a szerveroldali üzleti logikán át egészen a tesztelésig és a dokumentálásig.